Assignment : 15.5

Name : D .Akash

Ht.No : 2303A51422

Batch : 21

Task 1 – Student Attendance API

Task:

Create a RESTful API for student attendance tracking.

Instructions:

• Endpoints required:

o GET /attendance → View attendance records

o POST /attendance → Mark attendance

o PUT /attendance/{id} → Update attendance record

o DELETE /attendance/{id} → Remove attendance entry

• Ensure JSON-based request and response handling.

Expected Output:

• API for maintaining and updating attendance records.

Code :

```python
'''Task 1 – Student Attendance API
Task:
Create a RESTful API for student attendance tracking.
Instructions:
• Endpoints required:
o GET /attendance → View attendance records
o POST /attendance → Mark attendance
o PUT /attendance/{id} → Update attendance record
o DELETE /attendance/{id} → Remove attendance entry
• Ensure JSON-based request and response handling.
Expected Output:
• API for maintaining and updating attendance records.
• Proper handling of CRUD operations for attendance data.
• Use of appropriate HTTP status codes for responses.'''
# give code without flask
from datetime import datetime
class AttendanceRecord:
    def __init__(self, student_id, date, status):
        self.student_id = student_id
        self.date = date
        self.status = status
class AttendanceAPI:
    def __init__(self):
        self.attendance_records = []
        self.next_id = 1

    def get_attendance(self):
        return [record.__dict__ for record in self.attendance_records]

    def mark_attendance(self, student_id, date, status):
        record = AttendanceRecord(student_id, date, status)
        record.id = self.next_id
        self.attendance_records.append(record)
        self.next_id += 1
        return record.__dict__

```

```python
    def update_attendance(self, record_id, student_id=None, date=None, status=None):
        for record in self.attendance_records:
            if record.id == record_id:
                if student_id is not None:
                    record.student_id = student_id
                if date is not None:
                    record.date = date
                if status is not None:
                    record.status = status
                return record.__dict__
        return None

    def delete_attendance(self, record_id):
        for i, record in enumerate(self.attendance_records):
            if record.id == record_id:
                del self.attendance_records[i]
                return True
        return False
# Example usage
attendance_api = AttendanceAPI()
# Mark attendance
attendance_api.mark_attendance(student_id=1, date=str(datetime.now().date()), status='Present')
attendance_api.mark_attendance(student_id=2, date=str(datetime.now().date()), status='Absent')
# View attendance records
print(attendance_api.get_attendance())
# Update attendance record
attendance_api.update_attendance(record_id=1, status='Absent')
# View updated attendance records
print(attendance_api.get_attendance())
# Delete attendance record
attendance_api.delete_attendance(record_id=2)
# View final attendance records
print(attendance_api.get_attendance())
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SQL HISTORY    TASK MONITOR    QUERY RESULTS

PS C:\Users\akash\OneDrive\Desktop\training> & C:\Users\akash\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akash/OneDrive/Desktop/training/DSA.PY
 [{'student_id': 1, 'date': '2026-03-13', 'status': 'Present', 'id': 1}, {'student_id': 2, 'date': '2026-03-13', 'status': 'Absent', 'id': 2}]
 [{'student_id': 1, 'date': '2026-03-13', 'status': 'Absent', 'id': 1}, {'student_id': 2, 'date': '2026-03-13', 'status': 'Absent', 'id': 2}]
 [{'student_id': 1, 'date': '2026-03-13', 'status': 'Absent', 'id': 1}]
PS C:\Users\akash\OneDrive\Desktop\training>
```

Observations :

The API successfully manages student attendance records using RESTful endpoints. JSON requests and responses ensure smooth data exchange between client and server. Users can add, update, view, and delete attendance records efficiently. This system simplifies tracking and maintaining student attendance data.

Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management.

Instructions:

• Endpoints required:

o GET /inventory → List all items

o POST /inventory → Add new item

o PUT /inventory/{id} → Update stock quantity

o DELETE /inventory/{id} → Remove an item

• Implement validation for stock values.

Expected Output:

• API capable of managing inventory stock levels.

Code :

```python
'''Task 2 – Inventory Management API
Task:
Develop a REST API for warehouse inventory management.
Instructions:
• Endpoints required:
o GET /inventory → List all items
o POST /inventory → Add new item
o PUT /inventory/{id} → Update stock quantity
o DELETE /inventory/{id} → Remove an item
• Implement validation for stock values.
Expected Output:
• API capable of managing inventory stock levels.
• Proper error handling for invalid inputs and operations.
• Use of appropriate HTTP status codes for responses.'''
# give the code without flask or any framework, just the logic of the API
class InventoryItem:
    def __init__(self, item_id, name, quantity):
        self.item_id = item_id
        self.name = name
        self.quantity = quantity
# example inventory list
# this will act as our database for the sake of this example
# in a real application, this would be replaced with a database
# the key is the item_id and the value is the InventoryItem object
# this allows us to easily access items by their id
```

```python
inventory = {}
def get_inventory():
    return list(inventory.values())
def add_item(item_id, name, quantity):
    if item_id in inventory:
        return "Item already exists", 400
    if quantity < 0:
        return "Quantity cannot be negative", 400
    inventory[item_id] = InventoryItem(item_id, name, quantity)
    return "Item added successfully", 201
def update_stock(item_id, quantity):
    if item_id not in inventory:
        return "Item not found", 404
    if quantity < 0:
        return "Quantity cannot be negative", 400
    inventory[item_id].quantity = quantity
    return "Stock updated successfully", 200
def remove_item(item_id):
    if item_id not in inventory:
        return "Item not found", 404
    del inventory[item_id]
    return "Item removed successfully", 200
# Example usage:
print(add_item(1, "Item A", 10))  # Add new item
print(get_inventory())  # List all items
print(update_stock(1, 20))  # Update stock quantity
print(get_inventory())  # List all items after update
print(remove_item(1))  # Remove an item
print(get_inventory())  # List all items after removal
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR QUERY RESULTS

```
[Running] python -u "c:\Users\akash\OneDrive\Desktop\training\DSA.PY"
('Item added successfully', 201)
[<__main__.InventoryItem object at 0x0000021AC5594250>]
('Stock updated successfully', 200)
[<__main__.InventoryItem object at 0x0000021AC5594250>]
('Item removed successfully', 200)
[]
```

Observations :

The Inventory Management API efficiently handles warehouse stock operations through RESTful services. It

enables users to add new items, list all available items, update stock quantities, and remove items from the inventory. Proper validation ensures that stock values remain correct and prevents invalid data entry. This API helps maintain accurate inventory records and improves stock management.

Task 3 – Hotel Room Booking API

Task:

Develop a RESTful API for hotel room booking.

Instructions:

• Endpoints required:

o GET /rooms → View available rooms

o POST /rooms/book → Book a room

o GET /rooms/{id} → View booking details

o DELETE /rooms/{id} → Cancel booking

• Implement error handling for unavailable rooms.

Expected Output:

• API simulating hotel room booking operations.

Code :

```python
'''Task 3 - Hotel Room Booking API
Task:
Develop a RESTful API for hotel room booking.
Instructions:
• Endpoints required:
o GET /rooms → View available rooms
o POST /rooms/book → Book a room
o GET /rooms/{id} → View booking details
o DELETE /rooms/{id} → Cancel booking
• Implement error handling for unavailable rooms.
• Use in-memory data storage (e.g., a list or dictionary) to manage room bookings.
• Ensure the API is well-documented and follows RESTful principles.'''
# give code without flask or any other framework, just the logic of the API
class HotelRoomBookingAPI:
    def __init__(self):
        self.rooms = {
            1: {"type": "Single", "available": True, "booking_details": None},
            2: {"type": "Double", "available": True, "booking_details": None},
            3: {"type": "Suite", "available": True, "booking_details": None},
        }

    def get_available_rooms(self):
        available_rooms = {room_id: details for room_id, details in self.rooms.items() if details["available"]}
        return available_rooms

    def book_room(self, room_id, customer_name):
        if room_id not in self.rooms:
            return {"error": "Room not found."}

        if not self.rooms[room_id]["available"]:
            return {"error": "Room is already booked."}

        self.rooms[room_id]["available"] = False
        self.rooms[room_id]["booking_details"] = {"customer_name": customer_name}
        return {"message": f"Room {room_id} booked successfully."}

```

```python

    def get_booking_details(self, room_id):
        if room_id not in self.rooms:
            return {"error": "Room not found."}

        if self.rooms[room_id]["available"]:
            return {"error": "Room is not booked."}

        return self.rooms[room_id]["booking_details"]

    def cancel_booking(self, room_id):
        if room_id not in self.rooms:
            return {"error": "Room not found."}

        if self.rooms[room_id]["available"]:
            return {"error": "Room is not booked."}

        self.rooms[room_id]["available"] = True
        self.rooms[room_id]["booking_details"] = None
        return {"message": f"Booking for room {room_id} cancelled successfully."}
# Example usage
api = HotelRoomBookingAPI()
print(api.get_available_rooms())
print(api.book_room(1, "John Doe"))
print(api.get_booking_details(1))
print(api.cancel_booking(1))
print(api.get_available_rooms())

```

Output :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SQL HISTORY   TASK MONITOR   QUERY RESULTS
[Running] python -u "c:\Users\akash\OneDrive\Desktop\training\DSA.PY"
{1: {'type': 'Single', 'available': True, 'booking_details': None}, 2: {'type': 'Double', 'available': True, 'booking_details': None}, 3: {'type': 'Suite', 'available': True,
'booking_details': None}}
{'message': 'Room 1 booked successfully.'}
{'customer_name': 'John Doe'}
{'message': 'Booking for room 1 cancelled successfully.'}
{1: {'type': 'Single', 'available': True, 'booking_details': None}, 2: {'type': 'Double', 'available': True, 'booking_details': None}, 3: {'type': 'Suite', 'available': True,
'booking_details': None}}
```

Observations :

The Hotel Room Booking API simulates a basic hotel reservation system using RESTful endpoints. Users can view available rooms, book a room, check booking details, and cancel bookings when required. The API also includes error handling to prevent booking rooms that are already reserved. This system demonstrates how hotel booking operations can be managed efficiently using APIs**.**

Task 4 – Online Voting API

Task:

Create a RESTful API for an online voting system.

Instructions:

• Endpoints required:

o GET /candidates → List candidates

o POST /vote → Cast a vote

o GET /results → View voting results

• Ensure one vote per user using in-memory validation.

Expected Output:

• API simulating a secure online voting process.

Code :

```python
'''Task 4 - Online Voting API
Task:
Create a RESTful API for an online voting system.
Instructions:
• Endpoints required:
o GET /candidates → List candidates
o POST /vote → Cast a vote
o GET /results → View voting results
• Ensure one vote per user using in-memory validation.
Expected Output:
• API simulating a secure online voting process.
• Proper handling of user votes and accurate results display.
Note: This is a simplified version of an online voting system for demonstration purposes.
In a real-world application, you would need to implement additional security measures and use a persistent database.
'''
# give the code without flask or any other framework, just the logic of the API
class OnlineVotingAPI:
    def __init__(self):
        self.candidates = ["Alice", "Bob", "Charlie"]
        self.votes = {candidate: 0 for candidate in self.candidates}
        self.voters = set()  # To track voters and ensure one vote per user

    def get_candidates(self):
        return self.candidates

    def cast_vote(self, voter_id, candidate):
        if voter_id in self.voters:
            return "Error: You have already voted."

        if candidate not in self.candidates:
            return "Error: Invalid candidate."

        self.votes[candidate] += 1
        self.voters.add(voter_id)
        return "Vote cast successfully."

```

```python

    def get_results(self):
        return self.votes
# Example usage
if __name__ == "__main__":
    api = OnlineVotingAPI()

    print("Candidates:", api.get_candidates())

    print(api.cast_vote("user1", "Alice"))
    print(api.cast_vote("user1", "Bob"))  # Should return an error message
    print(api.cast_vote("user2", "Bob"))
    print(api.cast_vote("user3", "Charlie"))
    print("Results:", api.get_results())

```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SQL HISTORY    TASK MONITOR    QUERY RESULTS
[Running] python -u "c:\Users\akash\OneDrive\Desktop\training\DSA.PY"
Candidates: ['Alice', 'Bob', 'Charlie']
Vote cast successfully.
Error: You have already voted.
Vote cast successfully.
Vote cast successfully.
Results: {'Alice': 1, 'Bob': 1, 'Charlie': 1}

[Done] exited with code=0 in 0.585 seconds
```

Observations :

The Online Voting API provides a simple system for conducting digital voting. It allows users to view candidates, cast votes, and check the voting results through REST endpoints. The system ensures that each user can vote only once by using in-memory validation. This API demonstrates a basic approach to implementing a secure and fair online voting process.

Task 5 – Teacher Management API

Task:

Develop a RESTful API to manage teacher records.

Instructions:

• Endpoints required:

o GET /teachers → List all teachers

o POST /teachers → Add a new teacher

o PUT /teachers/{id} → Update teacher details

o DELETE /teachers/{id} → Remove a teacher

• Ensure proper error handling and JSON responses.

Expected Output:

• Functional API for managing teacher information.

Code :

```python
'''Task 5 - Teacher Management API
Task:
Develop a RESTful API to manage teacher records.
Instructions:
• Endpoints required:
o GET /teachers → List all teachers
o POST /teachers → Add a new teacher
o PUT /teachers/{id} → Update teacher details
o DELETE /teachers/{id} → Remove a teacher
• Ensure proper error handling and JSON responses.
Expected Output:
• Functional API for managing teacher information.
'''
# give the code without flask or any framework, just the logic of the API
class Teacher:
    def __init__(self, id, name, subject):
        self.id = id
        self.name = name
        self.subject = subject
class TeacherManagementAPI:
    def __init__(self):
        self.teachers = []
        self.next_id = 1

    def get_teachers(self):
        return [vars(teacher) for teacher in self.teachers]

    def add_teacher(self, name, subject):
        teacher = Teacher(self.next_id, name, subject)
        self.teachers.append(teacher)
        self.next_id += 1
        return vars(teacher)

```

```python
    def update_teacher(self, id, name=None, subject=None):
        for teacher in self.teachers:
            if teacher.id == id:
                if name:
                    teacher.name = name
                if subject:
                    teacher.subject = subject
                return vars(teacher)
        return None

    def delete_teacher(self, id):
        for i, teacher in enumerate(self.teachers):
            if teacher.id == id:
                del self.teachers[i]
                return True
        return False
# Example usage
api = TeacherManagementAPI()
# Add teachers
api.add_teacher("Alice", "Math")
api.add_teacher("Bob", "Science")
# Get all teachers
print(api.get_teachers())
# Update a teacher
api.update_teacher(1, name="Alice Smith")
# Get all teacher after update
print(api.get_teachers())
# Delete a teacher
api.delete_teacher(2)
# Get all teachers after deletion
print(api.get_teachers())
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SQL HISTORY    TASK MONITOR    QUERY RESULTS

PS C:\Users\akash\OneDrive\Desktop\training> & C:\Users\akash\AppData\Local\Programs\Python\Python313\pyt
[{'id': 1, 'name': 'Alice', 'subject': 'Math'}, {'id': 2, 'name': 'Bob', 'subject': 'Science'}]
[{'id': 1, 'name': 'Alice Smith', 'subject': 'Math'}, {'id': 2, 'name': 'Bob', 'subject': 'Science'}]
[{'id': 1, 'name': 'Alice Smith', 'subject': 'Math'}]
PS C:\Users\akash\OneDrive\Desktop\training>
```

Observations :

The Teacher Management API is designed to manage teacher information using RESTful operations. It allows users to add new teachers, view teacher details, update teacher records, and delete teacher information when necessary. The API also includes proper error handling and JSON-based responses for better communication. This system helps maintain organized and structured teacher records within an application.